

냉장고팀 기획제안서

프로젝트명: 양심냉장고

팀장: 이기훈

팀원 1: 김도형

팀원 2: 김영빈

팀원 3: 서지원

목차

1. 프로젝트 개요.....	4
1.1 문제정의 및 솔루션.....	4
1.2 주 타겟 및 Stakeholder.....	4
1.3 핵심 가치.....	4
2. 기존 사례 분석 및 사업화 가능성.....	6
2.1 유사 제품/서비스.....	6
2.1.1 예시.....	6
2.1.2 유사 서비스 분석 결과 및 차별점 제시.....	6
2.2 사업화 가능성.....	7
2.2.1 비즈니스 모델 정의.....	7
2.2.2 시장규모 분석.....	8
2.2.3 상용화 시 예상 사업규모.....	9
3. User Story Mapping.....	10
3.1 페르소나 정의.....	10
3.1.1 페르소나 1: 냉장고 사용자 '김아주'.....	10
3.1.2 페르소나 2: 냉장고 관리자 '이사감'.....	10
3.1.3 페르소나 3: 서비스 관리자 (Admin).....	10
3.2 사용자 여정 (User Journey).....	11
3.2.1 냉장고 사용자 Journey.....	11
3.2.2 냉장고 관리자 Journey.....	11
3.2.3 서비스 관리자 Journey.....	11
3.3 Epics & User Stories.....	12
3.3.1 Epic 1: 사용자 등록 및 로그인.....	12
3.3.2 Epic 2: 그룹 생성 및 참여.....	12

3.3.3 Epic 3: 냉장고 및 식품 관리.....	13
3.3.4 Epic 4: 장바구니 등록 및 삭제.....	13
3.3.5 Epic 5: 게시판 이용.....	13
3.3.6 Epic 6: 마이페이지.....	14
3.3.7 Epic 7: 그룹 설정.....	14
4. 설계 및 구현.....	15
4.1 기술 분석.....	15
4.1.1 기술 스택.....	15
4.1.2 개발 및 협업 환경.....	16
4.1.3 핵심 기술 및 알고리즘 (Optional).....	17
4.1.4 하드웨어 구축 기술 (Optional).....	17
4.2 시스템 설계.....	18
4.2.1 시스템 개요.....	19
4.2.2 소프트웨어 아키텍처.....	20
4.2.3 데이터 설계.....	23
4.2.4 API 정의.....	28
5. 과제 수행 계획.....	34
5.1 일정 계획.....	34
5.2 업무 분장.....	34
5.3 협업 및 DevOps 자동화.....	34
5.4 성과 측정 방안.....	35
5.4.1 KPI Tree.....	36
5.4.2 정량적 KPI.....	36
5.4.3 정성적 KPI.....	37

1. 프로젝트 개요

1.1. 문제 정의 및 솔루션

단체 생활을 하는 경우 공용 냉장고를 사용하는 경우가 빈번하다. 그러나, 혼자 사용하는 개인 냉장고와 달리 여러 사용자가 존재하는 공용 냉장고는 관리자 입장에서 몇 가지 불편함이 존재한다. 먼저, 냉장고 내부에 존재하는 전체 품목 파악이 어렵고, 각 식품의 소유자를 파악하기 어려워 장기간 방치되는 식품을 처리하기 곤란하다. 또한, 다수의 냉장고 사용자들에게 여러 공지사항들을 전달하는 데에도 어려움을 겪고 있다. 이러한 불편함은 다수의 냉장고를 관리하는 경우 배가 된다.

공용 냉장고 관리자의 불편함을 해결하고자, 냉장고 사용자들의 질서 있는 사용을 유도하는 서비스, '양심냉장고'를 제안한다. 관리자는 사용자들이 서비스 내에 등록한 품목들을 통해 냉장고 내부 및 각 품목의 소유자를 한 눈에 파악할 수 있고, 필요한 공지 및 알림을 손쉽게 전달 가능할 수 있다. 서비스 내에 구현된 규칙들은 사용자들로 하여금 쾌적한 냉장고 내부를 유지하도록 참여를 독려한다. 더불어 공용 냉장고를 단순한 사용자 또한 냉장고 속 자신의 식품을 파악하고 관리하기 쉬워진다는 부가 효과까지 누릴 수 있다.

1.2. 주 타겟 및 Stakeholder

본 서비스의 주요 타겟은 공용 냉장고를 관리하는 관리자(기숙사 사감, 호텔 관리인 등)이다.

생태계를 구성하는 구성원(Stakeholders)으로는 관리자와 사용자, 그리고 서비스 제공자를 설정할 수 있다.

1.3. 핵심 가치

본 프로젝트에서 설정한 생태계 구성원(Stakeholders)들은 각각 다음과 같은 가치를 갖게 된다.

- 관리자 : 공용 냉장고 사용자들과 식품들을 효율적으로 관리할 수 있다. 개인 또는 전체에게 전달해야 할 공지사항을 편리하게 전달 가능하다.
- 사용자 : 공용 냉장고에 본인의 상품을 등록하여 품목을 관리할 수 있다. 질서 있는

이용을 실천하며 쾌적한 냉장고 내부를 누릴 수 있다.

- 서비스 제공자 : 사용자들의 식재료 데이터를 확보하여 서비스를 확장하고 수익 모델을 창출할 수 있다.

본 서비스는 기존의 냉장고를 관리하는 다른 서비스들과 차별점을 갖는다. 먼저, 기존의 서비스들은 대부분이 개인의 냉장고 속 품목을 관리하는 수준에 지나지 않고, 그룹 간 공용 냉장고를 관리할 수 있는 서비스의 경우도 대등한 구성원끼리의 단순한 품목 공유에 불과하다. 양심 냉장고는 주타겟층인 '관리자'에 집중한다. 사용자들의 자발적 참여를 유도하는 기능을 통해 쾌적한 내부를 유지하는 데 중점을 뒀기에, 기존 서비스에서는 볼 수 없던 가치를 갖는다.

2. 기존 사례 분석 및 사업화 가능성

2.1. 유사 제품/서비스

2.1.1. 예시

- '원더프리지'
 - 개인 냉장고/패밀리 냉장고 및 참여자/관리자 구분한다.
 - QR 코드를 사용해 냉장고 멤버 초대 및 참여 가능하다.
 - 식품에 대한 소유권 표시와 멤버간 소통 창구가 존재하지는 않는다
 - 본 프로젝트와 가장 유사한 서비스이다.
- '냉장고 요정'
 - 냉장/냉동, 보관 기한을 입력하여 식품을 등록한다
 - 알림 기능 존재한다.
- '유통기한 언제지'
 - 냉장고 생성, 상품 등록, 친구 추가 등 다양한 기능이 존재한다.
 - 서비스 자체의 오류 및 버그 다수 보인다.
- '냉장고 메모장'
 - UI가 냉장고 형식으로 직관적으로 구현되었다.

2.1.2. 유사 서비스 분석 결과 및 차별점 제시

- 대부분의 서비스들이 개인 냉장고 관리를 중심으로, 단순 음식 등록 기능만을 제공한다.
 - '원더프리지'의 경우 냉장고를 개인/패밀리로 구분하여 관리하도록 한다.
 - 본 프로젝트의 경우 이러한 아이디어를 확장하여 다수의 인원이 다수의 냉장고를 사용하며, 음식을 등록한 후 보관기한이 초과되었을 때 음식 처리에

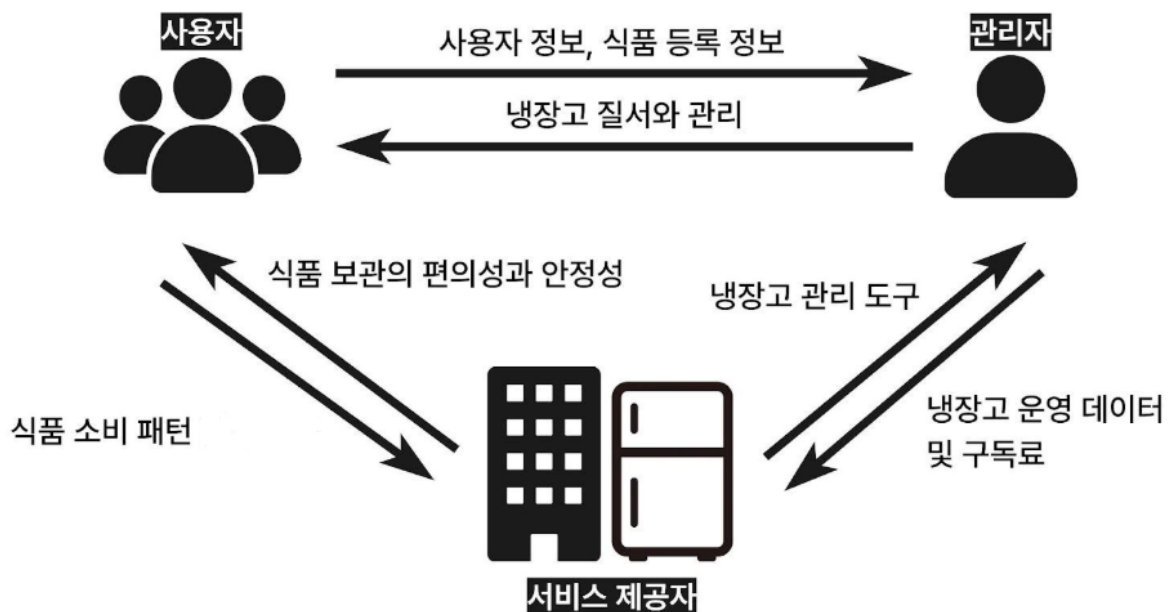
대한 방안을 함께 제공한다.

- '냉장고 요정'의 경우 알림 기능을 탑재하여 사용자에게 소비기한 알림을 보낸다.
 - 본 프로젝트에서는 단순 알림 수신뿐만 아니라 음식에 대한 상태를 어떻게 변환할 것인지 사용자가 답장할 수 있도록 하여 냉장고 내부 관리에 주체적으로 참여할 수 있는 환경을 마련한다.
- '냉장고 메모장'의 경우 UI를 냉장고로 구현하여 직관적인 느낌을 준다.
 - 본 프로젝트에서는 이러한 레퍼런스를 바탕으로, 냉장고 내부의 칸을 분리하여 그룹원 목록, 그룹 설정 등 다양한 기능을 배치함으로써 사용자 경험을 향상시킨다.

2.2. 사업화 가능성

2.2.1. 비즈니스 모델 정의

해당 비즈니스 모델은 공용 냉장고를 사용하는 관리자와 사용자를 대상으로 식품 등록 및 관리, 보관기한 알림 등을 지원하는 플랫폼으로, 관리자별 유료 구독, B2B 연계를 통한 수익 창출을 목표로 한다.



본 서비스는 관리자, 사용자, 서비스 제공자 간의 상호작용을 기반으로 운영되며, 각 주체는

데이터, 기능, 비용을 교환하는 구조를 형성한다.

- 사용자 → 관리자
 - 식품 등록 정보 및 사용자 정보를 제공한다.
 - 냉장고 이용 질서 유지에 참여한다.
- 사용자 → 서비스 제공자
 - 식품 소비 패턴 및 이용 데이터를 제공한다.
- 관리자 → 사용자
 - 냉장고 상태 관리 및 운영을 수행한다.
 - 체계적인 이용 환경을 제공한다.
- 관리자 → 서비스 제공자
 - 냉장고 운영 데이터를 제공한다.
 - 서비스 이용 구독료 지불을 지불한다.
- 서비스 제공자 → 사용자
 - 식품 보관 편의성을 제공한다.
- 서비스 제공자 → 관리자
 - 체계적인 냉장고 관리 도구를 제공한다.
 - 운영 효율성 개선을 지원한다.

2.2.2. 시장규모 분석

시장 규모

- 주요 타겟: 공용 냉장고를 사용하는 공동생활 인구
 - 기숙사: 한국사학진흥재단의 '대학별 기숙사 수용 현황' 2025 년도 통계자료에 따르면 국내 대학 392 개에서 기숙사 인원은 약 40 만명으로 측정되었다.
 - 고시원: 공공데이터포털의 '서울특별시 고시원 정보'에 따르면 연간 고시원 수는 약 5000 개(2024~2025 년도)이다. 고시원 한 곳당 인원을 30 명으로

가정했을 때, 거주 인원은 약 15 만명으로 추정된다.

- 중소기업: 서울 특별시에서 제공하고 있는 서울시 중소기업 종사자수 통계를 보면 2019 년도 기준 약 360 만명이 기업에 종사하고 있는 것으로 드러났다. 농업 및 어업, 광업, 건설업 등 비사무직 직군을 배제하면 약 130 만명의 사무직 종사자가 존재하는 것으로 추정된다.
- 주요 타겟층 외의 각종 세어하우스 및 공유오피스 등 공동 생활 인원을 전부 합산하면 해당 서비스의 잠재 규모 고객은 180 만명 이상이 될 것으로 추정된다. 유료 구독료를 지불할 관리자의 수는 전체의 3%인 5.4 만명으로 가정한다.
 - TAM(전체 시장): 5.4 만명(이상)
 - SAM(서비스 가능 시장): 전체 시장의 30% 가정할 경우 약 1.62 만명
 - SOM(초기 확보 시장): 유효 시장의 5%로 가정할 경우 약 810 명

사용자 성장세

- 초기: 810 명
- 성장기: 약 0.2 만명(SAM 의 약 12%)
- 중장기: 약 1 만명(SAM 의 약 60%)

서비스 이용 비용

- 하드웨어 장비 대여를 포함하여, 유료 구독 비용을 25,000 원으로 산정한다. 연 ARPU 는 300,000 원으로, 초기 하드웨어 장비 구입 비용은 한 패키지당 200,000 원, 설치 운반 등의 연간 장비 관리비는 50,000 원으로 설정해 계산을 진행한다.

2.2.3. 상용화 시 예상 사업 규모

- 초기: $810 \text{ 명} \times 300,000 \text{ 원} - (0 \text{ 명} - 810 \text{ 명}) \times 200,000 \text{ 원} - 810 \text{ 만명} \times 50,000 =$
약 4050 만원/년
- 성장기: $0.2 \text{ 만 명} \times 300,000 \text{ 원} - (0.2 \text{ 만명} - 810 \text{ 명}) \times 200,000 - 0.2 \text{ 만명} \times 50,000$
= 약 2.62 억 원/년
- 중장기: $1 \text{ 만 명} \times 300,000 \text{ 원} - (1 \text{ 만명} - 0.2 \text{ 만 명}) \times 200,000 - 1 \text{ 만명} \times 50,000 =$
약 9 억 원/년
- 최대 시장 매출: 약 9 억 원 규모로 산출 예상된다.

3. User Story Mapping

3.1. 페르소나 정의

3.1.1. 페르소나 1: 냉장고 사용자 '김아주'

- **설명:** 대학교 기숙사에 생활하면서 공용 냉장고를 사용하는 대학생이다. 체계적인 음식 보관 및 관리의 편의성을 원한다.
- **목표**
 - 내 음식의 보관 상태와 위치를 쉽게 확인한다.
 - 보관기간 관리 및 알림을 통한 음식 낭비 최소화한다.

3.1.2. 페르소나 2: 냉장고 관리자 '이사감'

- **설명:** 기숙사 사감으로, 공용 냉장고를 운영·관리하며 사용자 간 질서를 유지하고 효율적인 공간 활용을 책임진다.
- **목표**
 - 냉장고 내 식품 상태 파악한다.
 - 보관기간 초과 식품 및 방치된 음식을 정리한다.
 - 사용자 간 분쟁(도난, 자리 문제 등) 최소화한다.

3.1.3. 페르소나 3: 서비스 관리자 (Admin)

- **설명:** 플랫폼의 안정성과 공정성을 유지하며, 전체 서비스 운영 및 품질을 관리하는 담당자이다.
- **목표**
 - 사용자 및 냉장고를 관리한다.
 - 서비스 품질을 유지한다.

- 음식 구매 패턴, 음식 폐기율 등의 통계를 수집해 분석 데이터 기반 운영 정책 수립한다.

3.2. 사용자 여정 (User Journey)

3.2.1. 냉장고 사용자 Journey

- 1) 앱을 실행한다.
- 2) 원하는 그룹의 코드를 입력하여 그룹에 가입한다.
- 3) 그룹 내 냉장고를 선택한다.
- 4) 식품을 등록한다.
- 5) 식품의 보관 기간을 확인한다.
- 6) 공용음식과 폐기음식을 조회하여 공용음식을 섭취한다.

3.2.2. 냉장고 관리자 Journey

- 1) 앱을 실행한다.
- 2) 그룹 생성 및 사용자 초대를 수행한다.
- 3) 그룹에 냉장고를 추가 생성한다.
- 4) 냉장고 상태 및 폐기 대상 식품을 확인한다.
- 5) 문제 상황 발생 시 게시판에 공지사항을 작성한다.

3.2.3. 서비스 관리자 Journey

- 1) 신규 사용자에게 대한 모니터링을 수행한다.
- 2) 신고 접수를 처리하고, 계정에 제재를 가한다.
- 3) 사용자 활동 및 이용 패턴을 분석한다.
- 4) 서버 및 데이터를 관리한다.

5) 사용자 피드백을 수집하여 기능을 업데이트한다.

3.3. Epics & User Stories

3.3.1. Epic 1: 사용자 등록 및 로그인

User Story 1.1: 나는 사용자로서, 앱에서 회원가입을 하고 싶다 → 서비스를 이용하기 위한 계정을 생성하기 위해

User Story 1.2: 나는 사용자로서, 앱에 로그인하고 싶다 → 개인화된 서비스와 데이터를 이용하기 위해

User Story 1.3: 나는 사용자로서, 앱에서 로그아웃을 하고 싶다 → 개인 정보 보호 및 계정 보안을 유지하기 위해

3.3.2. Epic 2: 그룹 생성 및 참여

User Story 2.1: 나는 사용자로서, 그룹을 생성하고 관리자 권한을 갖고 싶다 → 공용 냉장고 환경을 직접 구성하고 관리하기 위해

User Story 2.2: 나는 관리자로서, 그룹 내 냉장고 수를 설정하고 싶다 → 실제 환경에 맞는 냉장고 구조를 반영하기 위해

User Story 2.3: 나는 사용자로서, 초대 코드를 통해 그룹에 참여하고 싶다 → 특정 공동생활 그룹에 쉽게 합류하기 위해

User Story 2.4: 나는 관리자로서, 냉장고의 이름을 설정하고 추가하고 싶다 → 냉장고를 구분하고 효율적으로 관리하기 위해

3.3.3. Epic 3: 냉장고 및 식품 관리

User Story 3.1: 나는 사용자로서, 식품을 냉장고에 등록하고 싶다 → 내가 보관 중인 식품을 체계적으로 관리하기 위해

User Story 3.2: 나는 사용자로서, 내가 등록한 식품을 삭제하고 싶다 → 섭취 완료 또는 폐기된 식품 정보를 정리하기 위해

User Story 3.3: 나는 사용자로서, 다른 사용자의 식품 정보를 확인하고 싶다 → 공용 냉장고 내 식품 현황을 파악하기 위해

User Story 3.4: 나는 사용자로서, 알림함을 통해 보관기한이 지난 상품에 대해 상태를 선택하고 싶다 → 식품의 안전성과 활용 여부를 판단하기 위해

User Story 3.5: 나는 사용자로서, 내가 응답하지 않은 경우 상품을 자동으로 폐기 처리하고 싶다 → 관리하지 못한 식품으로 인한 문제를 방지하기 위해

User Story 3.6 나는 사용자로서, 내 식품을 공용 음식으로 전환하고 싶다 → 다른 사용자와 식품을 나누기 위해

3.3.4. Epic 4: 장바구니 등록 및 삭제

User Story 4.1: 나는 사용자로서, 구매 예정 상품을 장바구니에 등록하고 싶다 → 구매 계획을 미리 정리하기 위해

User Story 4.2: 나는 사용자로서, 다른 사용자에게 공동 소유를 제안하고 싶다 → 식품을 함께 구매하여 비용을 절감하기 위해

User Story 4.3: 나는 사용자로서, 공동 소유 제안을 수락하거나 거절하고 싶다 → 내 소비 의사에 따라 참여 여부를 결정하기 위해

User Story 4.4: 나는 사용자로서, 장바구니의 상품을 냉장고로 이동시키고 싶다 → 실제 구매한 식품을 관리 시스템에 반영하기 위해

User Story 4.5: 나는 사용자로서, 장바구니 상품을 삭제하고 싶다 → 변경된 구매 계획을 반영하기 위해

3.3.5. Epic 5: 게시판 이용

User Story 5.1: 나는 사용자로서, 게시글을 작성/수정/삭제하고 싶다 → 공동구매 및 커뮤니티 활동에 참여하기 위해

User Story 5.2: 나는 사용자로서, 댓글을 작성 및 수정하고 싶다 → 다른 사용자와 의견을 교환하기 위해

User Story 5.3: 나는 사용자로서, 게시글에 추천을 하고 싶다 → 유용하거나 공감되는 글을 표현하기 위해

User Story 5.4: 나는 관리자로서, 공지 게시글을 작성/수정/삭제하고 싶다 → 중요한

정보를 구성원에게 전달하기 위해

User Story 5.5: 나는 관리자로서, 게시글 및 댓글을 삭제하고 싶다 → 부적절한 콘텐츠를 관리하기 위해

3.3.6. Epic 6: 마이페이지

User Story 6.1: 나는 사용자로서, 프로필 이미지를 설정 및 변경하고 싶다 → 최신 정보를 유지하기 위해

User Story 6.2: 나는 사용자로서, 알림 주기에 관한 상세 설정을 하고 싶다 → 서비스 이용 방식을 직접 조절하기 위해

User Story 6.3: 나는 사용자로서, 회원 탈퇴를 하고 싶다 → 서비스 이용을 중단하고 데이터를 정리하기 위해

3.3.7. Epic 7: 그룹 설정

User Story 7.1: 나는 사용자로서, 그룹 내 닉네임을 수정하고 싶다 → 그룹별로 나를 구분하기 쉽게 하기 위해

User Story 7.2: 나는 사용자로서, 그룹 구성원 목록을 확인하고 싶다 → 함께 사용하는 사람들을 파악하기 위해

User Story 7.3: 나는 사용자로서, 그룹에서 탈퇴하고 싶다 → 더 이상 해당 공동생활 환경에 속하지 않을 때 정리하기 위해

User Story 7.4: 나는 관리자로서, 구성원의 권한을 변경하거나 탈퇴시키고 싶다 → 그룹 운영을 효율적으로 관리하기 위해

User Story 7.5: 나는 관리자로서, 그룹 정보를 수정하고 싶다 → 그룹의 최신화된 정보를 반영하기 위해

4. 설계 및 구현

4.1. 기술 분석

4.1.1 기술 스택

React Native (Expo)

- **목적:** 모바일 애플리케이션 UI 구현 및 사용자 인터페이스 제공에 사용된다.
- **선택 이유:** 하나의 코드베이스로 iOS/Android 동시 개발이 가능하며, 컴포넌트 기반 구조로 유지보수성이 높다. Expo 를 통해 개발 및 빌드 환경을 단순화할 수 있다.

Spring Boot

- **목적:** 서버 API 구현, 백엔드 비즈니스 로직 처리에 사용된다.
- **선택 이유:** 빠른 개발이 가능하고 REST API 지원이 용이하며, 다양한 스프링 스타터를 통해 확장성이 높다.

Gradle

- **목적:** 의존성 관리 및 빌드 자동화에 사용된다.
- **선택 이유:** 빠른 빌드 속도와 유연한 설정을 지원하며 Spring Boot 와의 호환성이 우수하다.

MySQL

- **목적:** 관계형 데이터베이스 관리 및 데이터 저장에 사용된다.
- **선택 이유:** 안정성과 성능이 검증된 DBMS 로, Spring Boot 와의 연동이 용이하다.

JWT (JSON Web Token)

- **목적:** 사용자 인증 및 세션 관리에 사용된다.
- **선택 이유:** 서버 상태를 유지하지 않는 Stateless 인증 방식으로 확장성이 뛰어나며 모바일 환경에서도 효율적으로 사용 가능하다.

FCM (Firebase Cloud Messaging)

- **목적:** 보관기한 임박, 공지 등 푸시 알림 전송에 사용된다.
- **선택 이유:** 모바일 앱에서 안정적인 푸시 알림 전송을 지원하며, 백그라운드 상태에서도 알림 수신이 가능하다.

MQTT (Message Queuing Telemetry Transport)

- **목적:** 백엔드 서버와 라즈베리파이 간의 실시간 비동기 메시지 통신에 사용된다.
- **선택 이유:** HTTP 요청보다 훨씬 가볍고 빠르며, 네트워크가 불안정한 IoT 환경에서도 메시지 전송 신뢰성을 보장할 수 있다.

4.1.2 개발 및 협업 환경

Git / GitHub

- **목적:** 소스 코드 버전 관리 및 협업에 사용된다.
- **활용 방식:** Git flow 기반 브랜치 전략을 사용하고 Pull Request(PR)를 통해 코드 리뷰를 진행한다.

CI/CD (GitHub Actions)

- **목적:** 자동 빌드 및 배포에 사용된다.
- **활용 방식:** 코드 푸시 시 GitHub Actions 를 통해 백엔드와 모바일 앱 각각에 대한 빌드 및 배포 파이프라인을 구성한다.

Docker

- **목적:** 백엔드 실행 환경 통일 및 컨테이너 기반 배포에 사용된다.
- **활용 방식:** Spring Boot 애플리케이션을 Docker 이미지로 빌드하여 EC2 환경에서 동일하게 실행한다.

Amazon EC2

- **목적:** 백엔드 서버 배포 및 운영에 사용된다.
- **활용 방식:** Docker 기반 Spring Boot 애플리케이션 실행 환경을 제공한다.

Amazon S3

- **목적:** 이미지 및 사용자 업로드 파일 저장에 사용된다.

- **활용 방식:** 식품 이미지, 프로필 이미지 등 파일을 저장하고 서버에서 접근하여 제공한다.

SSH & VNC (Virtual Network Computing)

- **목적:** 라즈베리파이 원격 접속 및 디바이스 환경 설정에 사용된다.

4.1.3 핵심 기술 및 알고리즘

React Native Component Architecture

- 모바일 UI 를 컴포넌트 단위로 구성하여 재사용성과 유지보수성을 향상한다.

Spring Boot Security + JWT

- JWT 기반 인증 및 권한 관리 구조를 통해 안전한 사용자 인증 시스템을 구현한다.

REST API 설계

- 클라이언트와 서버 간의 표준화된 HTTP 기반 통신 구조를 적용한다.

소비기한 기반 우선순위 알고리즘

- 식품의 소비기한까지 남은 시간을 기준으로 우선순위를 계산하여, 사용자에게 우선 소비가 필요한 식품을 추천한다.

식품 카테고리 분류 알고리즘

- 식품의 종류 및 특성을 기반으로 카테고리를 자동 분류하여 관리 효율성을 향상한다.

4.1.4 하드웨어 구축 기술

Bluetooth Label Printer

- **목적:** 식품 등록자 및 보관 시작일시 등이 기재된 실물 라벨 출력에 사용된다.
- **선택 이유:** 유지비가 적게 드는 감열식 블루투스 프린터를 채택하여 공용 공간에서의 관리 효율성을 극대화한다.

Raspberry Pi (IoT Gateway)

- **목적:** 백엔드 서버와 블루투스 라벨 프린터 사이의 중간 통신 게이트웨이 역할을

수행한다.

- **선택 이유:** 저렴한 비용으로 상시 전원을 유지하며 로컬 네트워크와 블루투스 통신을 동시에 지원할 수 있어, 다수 사용자가 매번 프린터와 직접 페어링해야 하는 불편함을 해소할 수 있다.

Zero-UI 기반 컨텍스트 라우팅 (NFC Deep Linking)

- 사용자가 NFC 스티커를 스캔하면 앱이 커짐과 동시에 특정 냉장고의 파라미터를 인증과 인쇄 명령을 처리하는 자동화 파이프라인이다.

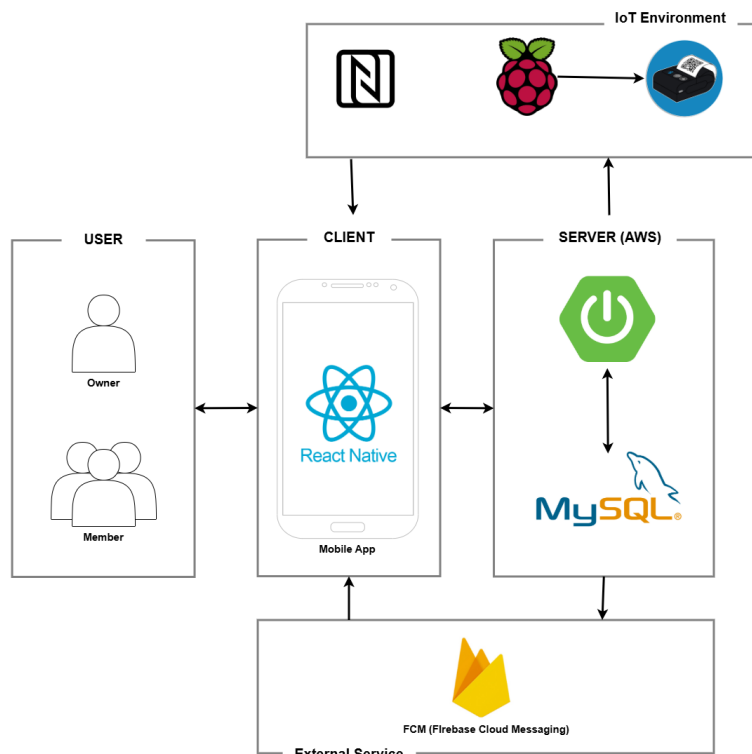
투스텝(Two-Step) 지연 등록 및 비동기 출력 구조

- 냉장고 앞에서의 물리적 대기 시간 최소화 및 사용자 편의성을 극대화한다.
- 동작 방식
 - 1. 사용자가 NFC 스티커를 태그하면, 서버가 식별자(UUID), 소유자 이름, 생성 시각만을 포함한 '임시 상태(PENDING)'의 식품 레코드를 DB 에 생성한다.
 - 2. 서버는 즉시 MQTT Broker 를 통해 라즈베리파이로 해당 기초 데이터(예: ID: 123, 등록자: 김아주, 시간: 04/08 15:20)의 인쇄 명령을 보내 라벨을 우선 출력한다.
 - 3. 이후 사용자가 앱 화면에서 실제 식품명과 수량 등 상세 정보를 입력하여 제출하면, 서버가 기존에 생성된 식별자(UUID)를 기반으로 DB 레코드를 업데이트하여 데이터를 완성한다.

4.2. 시스템 설계

4.2.1. 시스템 개요

- 시스템 구조도



USER

- 서비스를 사용하는 사용자로, 그룹을 생성 및 관리하는 Owner 와 그룹에 소속되어 식품 등록, 조회, 삭제 등의 기능을 이용하는 Member 로 구성된다.

CLIENT

- 사용자 인터페이스를 제공하며, 서버와 통신하여 데이터를 조회 및 조작하는 모바일 애플리케이션이다. React Native 와 Expo 를 기반으로 개발된다.

SERVER - Spring Boot

- 사용자 요청을 처리하고 비즈니스 로직을 수행하는 백엔드 API 서버이다.

SERVER – MySQL

- 사용자, 그룹, 식품 정보 등 서비스 데이터를 저장하고 관리하는 관계형 데이터베이스이다.

SERVER – AWS

- 백엔드 애플리케이션과 데이터베이스가 배포 및 운영되는 클라우드 인프라 환경이다.

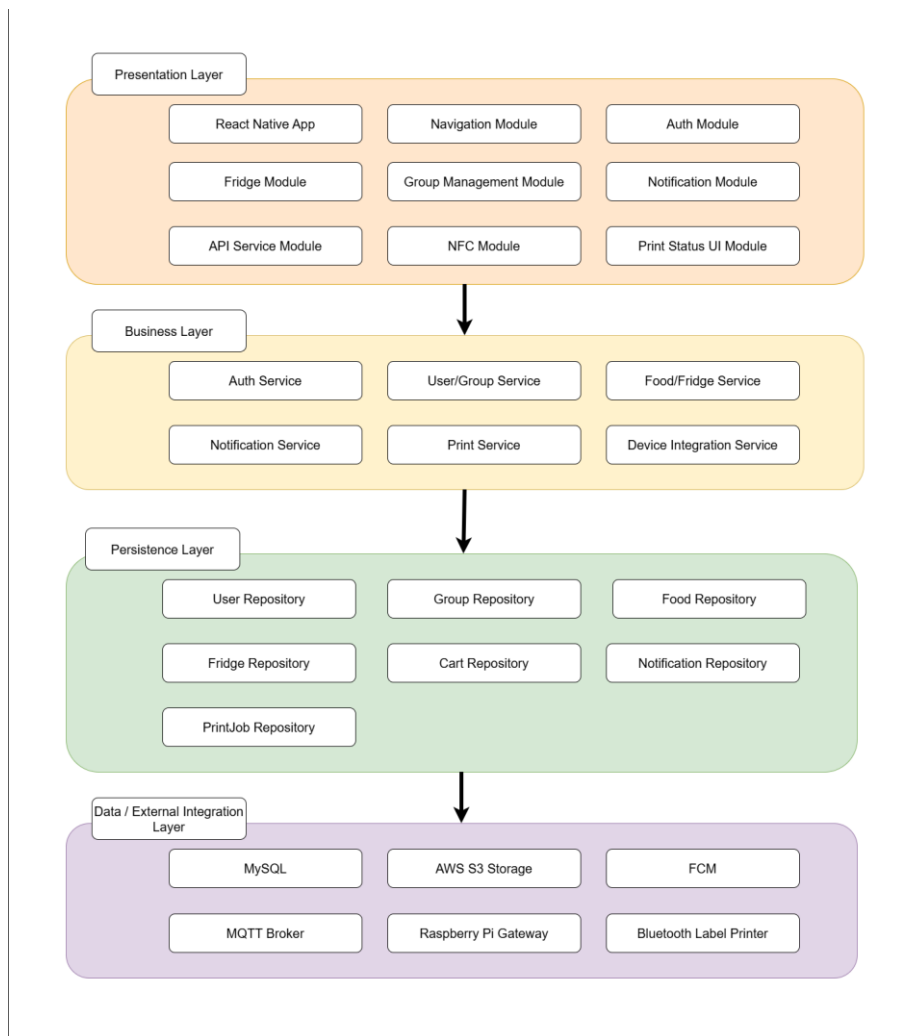
External Service - Firebase (FCM)

- 푸시 알림 기능을 지원하는 외부 서비스로, 보관기한 임박 알림, 공지 알림 등 서비스 내 주요 알림을 사용자에게 전달한다.

IoT Environment

- 서버와 연동하여 라벨 출력 기능을 수행하는 하드웨어 실행 환경으로, 메시지 브로커, 라즈베리파이, 블루투스 라벨 프린터로 구성된다.

4.2.2. 소프트웨어 아키텍처



- 컴포넌트 구조 정의
 - Presentation Layer

- ◆ App (Root Component) : 모바일 애플리케이션의 진입점으로, 전체 화면 구조와 전역 상태를 관리한다.
- ◆ Navigation Module : 화면 간 이동 및 내비게이션 흐름을 관리한다.
- ◆ Auth Module : 로그인/회원가입 처리, JWT 저장 및 인증 상태 관리
- ◆ Fridge Module : 식품 목록 조회, 등록, 삭제 기능 제공
- ◆ Group Management Module : Owner 중심으로 그룹 생성, 그룹 정보 관리, 멤버 관리 기능을 제공한다.
- ◆ Notification Module : 알림 목록 조회 및 UI 표시 기능을 제공한다.
- ◆ API Service Module : 백엔드와의 HTTP 통신 담당한다.
- ◆ NFC Module : NFC 태그 인식 및 특정 냉장고 컨텍스트로의 진입 기능을 제공한다.
- ◆ Print Status UI Module : 출력 요청, 출력 완료, 실패 상태를 사용자에게 표시한다.

■ Business Layer

- ◆ Auth Service : 사용자 인증, JWT 발급 및 검증 기능을 담당한다.
- ◆ User/Group Service : 사용자 정보 관리, 그룹 생성, 그룹 정보 조회 및 멤버 관리 기능을 담당한다.
- ◆ Food/Fridge Service : 식품 등록, 조회, 삭제 및 냉장고 관련 핵심 비즈니스 로직을 처리한다.
- ◆ Notification Service : 알림 생성 및 FCM 연동을 통한 푸시 알림 전송을 담당한다.
- ◆ Print Service : 식품 등록 시 라벨 출력 요청 생성, 출력 상태 관리, 재출력 기능을 담당한다.
- ◆ Device Integration Service : MQTT, Raspberry Pi, 블루투스 라벨 프린터 등 외부 장치와의 연동 로직을 담당한다.

■ Persistence Layer

- ◆ User Repository : 사용자 데이터 접근 및 CRUD 를 담당한다.
- ◆ Group Repository : 그룹 데이터 접근 및 CRUD 를 담당한다.

- ◆ Food Repository : 식품 데이터 접근 및 CRUD 를 담당한다.
- ◆ Fridge Repository : 냉장고 데이터 접근 및 CRUD 를 담당한다.
- ◆ Cart Repository : 장바구니 데이터 접근 및 CRUD 를 담당한다.
- ◆ Notification Repository : 알림 데이터 접근 및 CRUD 를 담당한다.
- ◆ PrintJob Repository : 라벨 출력 요청 이력과 상태 데이터 접근을 담당한다.

■ Data / External Integration Layer

- ◆ MySQL : 사용자, 그룹, 식품, 알림, 출력 요청 등 서비스 운영 데이터를 저장한다.
- ◆ AWS S3 Storage : 식품 이미지, 프로필 이미지 등 업로드 파일을 저장한다.
- ◆ Firebase Cloud Messaging : 사용자 디바이스에 푸시 알림을 전송한다
- ◆ MQTT Broker : 백엔드와 라즈베리파이 간의 비동기 메시지 송수신을 담당한다.
- ◆ Raspberry Pi Gateway : 서버의 출력 명령을 수신하여 블루투스 라벨 프린터에 전달하는 중간 장치 역할을 수행한다.
- ◆ Bluetooth Label Printer Interface : 등록자, 생성 시각, 식별자 등의 정보를 포함한 라벨을 실물로 출력한다.

● 사용 SW 패턴

- Component-Based Architecture (React Native)
 - ◆ UI 를 재사용 가능한 화면 및 컴포넌트 단위로 구성한다.
- RESTful API Design
 - ◆ 클라이언트와 서버 간의 표준화된 HTTP 기반 통신 구조를 설계한다.
- Layered Architecture
 - ◆ Presentation Layer, Business Layer, Persistence Layer, Data/External Integration Layer 로 역할과 책임을 분리하여 유지보수성과 확장성을

높인다.

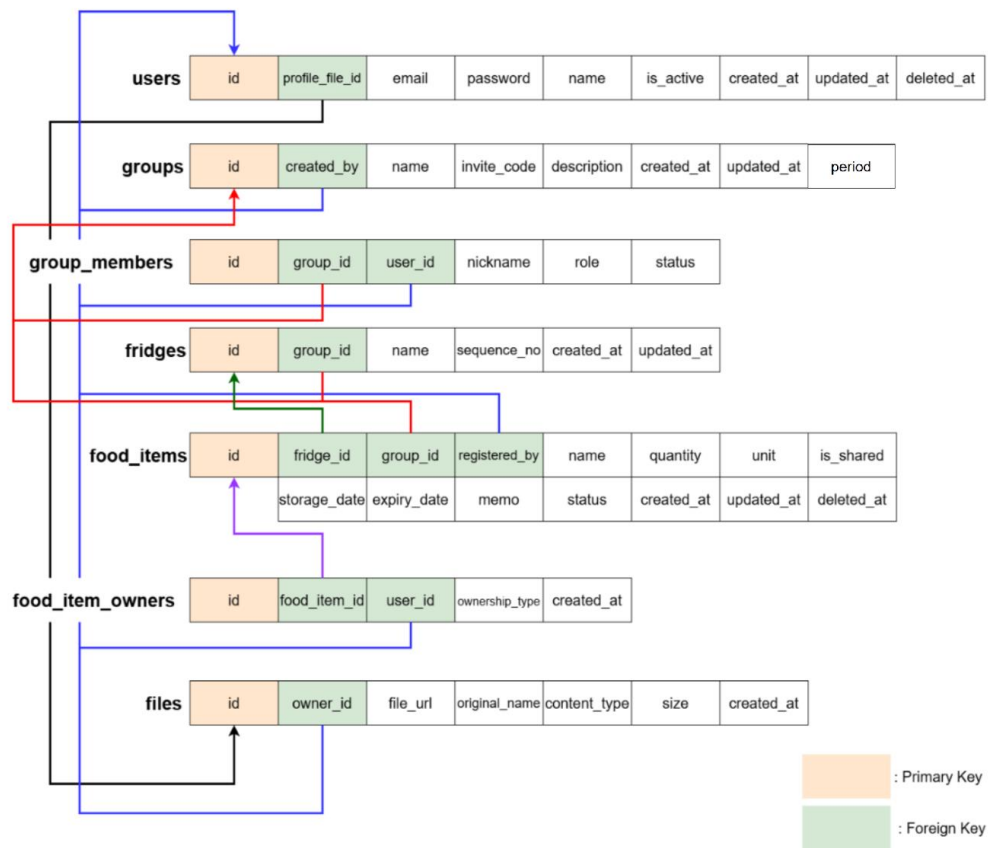
- **각 컴포넌트 역할**

- React Native / Expo : 모바일 사용자 인터페이스를 제공하고 백엔드 API 와 통신한다.
- Spring Boot : 비즈니스 로직을 처리하고 데이터 및 외부 서비스 연동을 담당한다.
- MySQL : 사용자, 그룹, 식품, 알림 등 서비스 운영 데이터를 저장한다.
- AWS S3 : 식품 이미지, 프로필 이미지 등 업로드 파일을 저장한다.
- FCM : 서버에서 생성된 알림을 사용자 디바이스로 전달한다.
- MQTT Broker : 백엔드와 라즈베리파이 간의 비동기 메시지 송수신을 담당한다.
- Raspberry Pi Gateway : 서버의 출력 명령을 수신하여 블루투스 라벨 프린터와 연결되는 IoT 게이트웨이 역할을 수행한다.
- Bluetooth Label Printer : 식품 식별 정보를 포함한 라벨을 실물로 출력한다.

4.2.3. 데이터 설계

본 시스템은 사용자, 그룹, 냉장고, 식품 관리, 게시판, 알림 등의 다양한 기능으로 구성되어 있으며, 전체 데이터베이스는 다수의 엔티티로 설계되어 있다. 본 제안서에는 그 중 핵심 기능인 사용자 관리, 그룹 생성 및 관리, 냉장고 기반 식품 관리 기능을 중심으로 주요 7 개의 엔티티를 선별하여 제시한다. 이후 제시되는 테이블 구조 및 상세 설명은 서비스의 핵심 흐름을 이해하기 위한 대표적인 데이터 모델을 기준으로 작성되었다.

- Entity Relationship Diagram



- 테이블 정의

users

Attribute	Type	Description	예시
id	BIGINT	(PK) 사용자 고유 ID	1
profile_file_id	BIGINT	(FK → files.id) 사용자의 프로필 이미지	1
email	VARCHAR	사용자 이메일 (고유)	abc@naver.com
password	VARCHAR	암호화된 비밀번호	hashed_pw
name	VARCHAR	사용자 이름	김아주
is_active	BOOLEAN	계정 활성 여부	true
created_at	DATETIME	생성 시각	2026-03-30
updated_at	DATETIME	수정 시각	2026-03-30
deleted_at	DATETIME	삭제 시각	NULL

groups

Attribute	Type	Description	예시
id	BIGINT	(PK) 그룹 고유 ID	3
created_by	BIGINT	(FK → users.id) 그룹을 생성한 사용자 ID	1
name	VARCHAR	그룹 이름	office1
invite_code	VARCHAR	초대 코드	ABC123
description	TEXT	그룹 설명	회사 냉장고
created_at	DATETIME	생성 시각	2026-03-30
updated_at	DATETIME	수정 시각	2026-03-30

group_members

Attribute	Type	Description	예시
id	BIGINT	(PK) 그룹원 고유 ID	10
group_id	BIGINT	(FK → groups.id) 그룹 ID	3
user_id	BIGINT	(FK → users.id) 사용자 ID	1
nickname	VARCHAR	그룹 내 닉네임	bean
role	ENUM	그룹원의 권한 OWNER / MEMBER	OWNER
status	ENUM	그룹원의 상태(활성/나감/방출) ACTIVE / LEFT / KICKED	ACTIVE

fridges

Attribute	Type	Description	예시
id	BIGINT	(PK) 냉장고 고유 ID	1
group_id	BIGINT	(FK → groups.id) 소속 그룹 ID	3
name	VARCHAR	냉장고 이름	1번 냉장고
sequence_no	INT	정렬용 번호	1
created_at	DATETIME	생성 시각	2026-03-30
updated_at	DATETIME	수정 시각	2026-03-30

food_items

Attribute	Type	Description	예시
id	BIGINT	(PK) 식품 고유 ID	12
fridge_id	BIGINT	(FK → fridges.id) 식품이 저장되는 냉장고 ID	1
group_id	BIGINT	(FK → groups.id) 소속 그룹 ID	3
registered_by	BIGINT	(FK → users.id) 식품 저장한 사람의 ID	1
name	VARCHAR	식품명	우유
quantity	INT	수량	3
unit	VARCHAR	단위	개, 봉지, ml 등
is_shared	BOOLEAN	공동 소유 여부	true
storage_date	DATE	보관 시작일	2026-03-01
expiry_date	DATE	소비기한	2026-03-10
memo	TEXT	메모	저지방
status	ENUM	상품의 상태(보관중/먹음/버림(관리자에 의해)/삭제 IN_FRIDGE / CONSUMED / DISCARDED / DELETED	IN_FRIDGE
created_at	DATETIME	생성 시각	-
updated_at	DATETIME	수정 시각	-
deleted_at	DATETIME	삭제 시각	NULL

food_item_owners

Attribute	Type	Description	예시
id	BIGINT	(PK) 식품 소유자 고유 ID	1
food_item_id	BIGINT	(FK → food_items.id) 해당 식품 ID	12
user_id	BIGINT	(FK → users.id) 소유자의 유저 ID	2
ownership_type	ENUM	소유자 유형(소유자/공동소유자) OWNER / CO_OWNER	CO_OWNER
created_at	DATETIME	생성 시각	-

files

Attribute	Type	Description	예시
id	BIGINT	(PK)	1
owner_id	BIGINT	(FK → users.id) 소유자	1
file_url	VARCHAR	저장 경로	/uploads/a.png
original_name	VARCHAR	원본 이름	milk.png
content_type	VARCHAR	파일 타입	image/png
size	BIGINT	파일 크기	102400
created_at	DATETIME	생성 시각	-

4.2.4. API 정의

회원가입, 로그인

Method	PATH	Message Format	Description
POST	/register	{email:String, NOT NULL password:String, NOT NULL name:String, NOT NULL isActive:boolean, default True createdAt:Date, NOT NULL}	회원가입 정보 전송
POST	/login	{email:String, NOT NULL, password:String, NOT NULL}	로그인 정보 전송
PUT	/mypage	{name:String, NOT NULL profileImg:String}	마이페이지에서 내 정보 수정

POST	/auth/findPW	{email:String, NOT NULL}	이메일로 계정 찾기
GET	/mypage		내 정보 불러오기
DELETE	/mypage		회원 탈퇴

그룹 관련 api

Method	PATH	Message Format	Description
GET	/groups	X	내가 속한 그룹 조회
POST	/groups	{groupName:String, NOT NULL nickname:String, NOT NULL inviteCode:String, NOT NULL description:String period:Date, default 14 createdBy:String, NOT NULL createdAt:Date, NOT NULL}	그룹 생성
PUT	/groups/{groupId}	{groupName:String, NOT NULL description:String period:Date}	그룹 정보 수정(관리자)

DELETE	/groups/{groupId}	X	그룹 삭제
GET	/groups/{groupId}/members	X 또는 ?nickname=String	그룹 내 멤버 조회(필터 가능)
POST	/groups/{groupId}/members	{userId:String, NOT NULL nickname:String, NOT NULL role:ENUM, ADMIN/MEMBER joinedAt:Date, NOT NULL}	그룹에 멤버로 가입 요청
PUT	/groups/{groupId}/members/{memberId}	{role:ENUM, ADMIN/MEMBER}	그룹 내 특정 멤버 권한 수정(관리자)
DELETE	/groups/{groupId}/members	X(전체 삭제) 또는 members: [{memberId:String, Not NULL}] (선택삭제)	그룹 내 멤버 삭제(관리자)
PUT	/groups/{groupId}/members/me	{nickname:String, NOT NULL}	그룹 내 닉네임 변경
DELETE	/groups/{groupId}/members/me	X	그룹 탈퇴

Method	PATH	Message Format	Description
GET	/groups/{groupId}/fridges	X 또는 ?fridgeName=String	그룹 내 냉장고 조회(필터 가능)
POST	/groups/{groupId}/fridges	{fridgeName:String createdBy:String createdAt:Date}	냉장고 추가 정보 전송
DELETE	/groups/{groupId}/fridges	X(전체 삭제) 또는 fridges: {[fridgeId:String, Not NULL]} (선택 삭제)	냉장고 삭제 요청
PUT	/groups/{groupId}/fridges/{fridgeId}	{fridgeName:String}	냉장고 이름 변경
GET	/groups/{groupId}/users/{userId}/foods		특정 그룹 내 해당 사용자의 음식 목록 조회
POST	/groups/{groupId}/users/{userId}/foods	{foodId:String fridgeId:String quantity:int memo:String status:ENUM("IN_FRIDGE"/"CONSUMED"/"DISCARDED"/"DELETED"/"CANDIDED"/"DISPOSED"), default "IN_FRIDGE" owners: [{userId:String role:ENUM("OWNER"/"SHARED")}] }	특정 그룹 내 해당 사용자의 냉장고에 음식 추가
DELTE	/groups/{groupId}/users/{userId}/foods	?X(전체 삭제) 또는 foods: {[foodId:String, Not NULL]} (선택 삭제)	특정 그룹 내 해당 사용자의 음식 삭제
PUT	/groups/{groupId}/users/{userId}/foods/{foodId}	{fridgeId:String quantity:int memo:String status:ENUM("IN_FRIDGE"/"CONSUMED"/"DISCARDED"/"	특정 그룹 내 해당 사용자의 음식 수정

		DELETED"/"CANDIDED"/"DISPOSED") owners: [{userId:String, role:ENUM, "OWNER"/"SHARED"}]}	
GET	/groups/{groupId}/food	X	그룹 내 모든 음식 조회
GET	/groups/{groupId}/food/{foodId}	X	그룹 내의 특정 음식 상세 조회
GET	/groups/{groupId}/fridges/{fridgeId}/foods	?status="CANDIDATE"	그룹 내 특정 냉장고 기한 지난(폐기 후보) 음식 목록 조회
GET	/groups/{groupId}/fridges/{fridgeId}/foods	?status="PUBLIC"	그룹 내 특정 냉장고 공용 음식 목록 조회
GET	/groups/{groupId}/fridges/{fridgeId}/foods	?status="DISPOSED"	그룹 내 특정 냉장고 기한 지난 폐기 음식 목록 조회

게시판 관련 api

Method	PATH	Message Format	Description
POST	/posts	{userId:String, NOT NULL groupId:String, NOT NULL title:String, NOT NULL	게시판 글 작성

		context:String, NOT NULL postType:ENUM, NOTICE/NORMAL createdAt:Date, NOT NULL}	
GET	/posts		게시글 목록 조회
DELETE	/posts/{postId}	{postId:String, NOT NULL}	게시판 글 삭제 요청
PUT	/posts/{postId}	{postId:String, NOT NULL userId:String, NOT NULL title:String, NOT NULL context:String, NOT NULL createdAt:Date, NOT NULL}	게시판 글 수정
GET	/posts/{postId}		게시글 상세조회
GET	/posts/{postId}/comments		댓글목록조회
POST	/posts/{postId}/comments	{postId:String, NOT NULL userId:String, NOT NULL context:String, NOT NULL createdAt:Date, NOT NULL}	댓글 작성
PUT	/comments/{commentId}	{commentId:String, NOT NULL userId:String, NOT NULL context:String, NOT NULL updatedAt:Date, NOT NULL}	댓글 수정
DELETE	/comments/{commentId}		댓글 삭제

POST	/posts/{postId}/like	{userId:String, NOT NULL}	댓글 좋아요
------	----------------------	---------------------------	--------

5. 과제 수행 계획

5.1. 일정 계획

번다운 차트를 통해 스프린트로 계획을 구상하였다.

5.2. 업무 분장

개발 업무 분장은 번다운 차트를 통해 할당하였다. 비개발 업무 분장은 아래와 같다.

요구사항 분석 및 구체화

- 사용자 관점에서의 핵심 기능을 도출하고 우선순위를 선정한다.

와이어프레임 설계 및 브랜딩

- Figma 를 사용하여 프로토타입을 제작하며, 서비스 로고 구상을 통해 프론트엔드 개발에 대한 가이드라인을 제시한다.

사용성 평가를 통한 품질 관리

- 잠재적 사용자 피드백을 수집하여 인터페이스의 직관성 및 서비스 접근성에 대한 개선안을 제시한다.

문서화 및 발표 준비

- 개발 중 작성되는 산출물을 작성하고, 발표를 위한 PPT 제작 및 주요 시나리오를 따른 서비스 시연을 구상한다.

5.3. 협업 및 DevOps 자동화

코딩 컨벤션

- 변수명, API 명은 첫 글자는 소문자, 중간 단어의 첫 글자는 대문자로 작성한다.
- 컴포넌트는 모든 단어의 첫글자를 대문자로 작성한다.

코드 리뷰 방식

- 모든 개발 작업은 작업 완료 후, Pull Request 를 생성해 최소 2 명 이상의 팀원으로부터 승인을 받아야만 상위 브랜치에 병합 가능할 수 있는 체계를 운영하여 코드 결함을 최소화한다.

테스트 자동화 방식 및 도구 설명

- 단위 테스트 : Java/Spring Boot 환경에서 기본으로 사용하는 JUnit 테스트를 사용하여 로직들의 정확성을 검증한다.
- 자동화 연동 : 테스트 코드는 로컬 환경 뿐 아닌, CI 파이프라인 GitHub Actions 내에서 자동으로 실행되도록 설정하여 테스트를 통과하지 못한 코드는 배포되지 않도록 차단한다.

깃 브랜치 활용 방식

- main : 안정 버전으로 무결점을 유지한다.
- develop : 개발 버전으로 개발 기능별로 브랜치를 나누어 단위 개발에 집중한다.

데브옵스 파이프라인 구성

- CI : 개발자가 GitHub 에 코드를 푸시하면 자동으로 빌드와 JUnit 테스트를 수행해 코드의 무결성을 검증한다
- Containerization : 검증된 소스 코드를 Docker 이미지로 빌드한다.
- CD : 빌드된 이미지를 Docker Hub 에 업로드 한 후, Amazone EC2 서버에서 최신 이미지를 pull 하여 컨테이너를 재구동한다.

5.4. 성과 측정 방안

5.4.1. KPI Tree



5.4.2. 정량적 KPI

- 공용 공간 청결도 : 90% 이상. 냉장고 내부 품목 중 폐기 대상으로 분류된 식품이 차지하는 비중을 역산하는 방식으로, $\{1 - (\text{폐기 대상 상태의 품목 수} / \text{냉장고 내 전체 품목 수})\} \times 100$ 을 계산해 도출한다.
- 기한 초과 알림 응답률 : 80% 이상. 기한 초과 알림 건수 중 자동 폐기 대상 처리 전에 직접 응답한 비중을 $(\text{사용자가 답변한 알림 수} / \text{발송된 전체 알림 수}) \times 100$ 를 계산해 도출한다.
- 공유 음식 전환 성공률 : 30% 이상. 보관 기한을 넘긴 식품 중 소유자의 답변에 의해 성공적으로 공유 음식으로 전환 된 비중을 $(\text{공유 음식 전환 수} / \text{총 폐기 대상 품목 수}) \times 100$ 을 계산해 도출한다.
- API 응답 속도 : 평균 300ms 이하. JMeter 의 Summary Report 를 추가해 실행시켜

‘Average’ 또는 ‘90% Line’ 수치를 통해 평가한다.

- 알림 발송 성공률 : 99% 이상. Firebase Console 의 ‘Cloud Messaging’-‘Reports’으로 기간을 설정해 발송 대비 전송 성공 그래프를 확인한다. 대시보드의 출력된 성공률로 직접적인 확인을 거친다.

5.4.3. 정성적 KPI

- 사용자 경험을 아래 보기들을 포함한 객관식 질문(1~4)를 통해 설문조사를 진행하여 평가한다.
 - 1 점 : 그러한 느낌은 전혀 없고 오히려 평소보다 불편함을 느낀다.
 - 2 점 : 그런 느낌이 들긴 하나, 미미하여 해당 서비스 이용 전과 별 차이가 없다.
 - 3 점 : 그런 느낌을 분명히 받는다.
 - 4 점 : 그런 느낌을 받기에 부족함 없는 서비스이다.
 - 5 점 : 그런 느낌 덕분에 서비스를 애용하며 다른 이에게도 서비스를 추천하고 싶다.
- 질문 1) 사용자는 앱을 통해 냉장고 현황을 실시간으로 파악할 수 있어, 실물 냉장고 상태에 대한 데이터 신뢰감을 느낀다.
- 질문 2) 보관 기한 초과 시 폐기할지 공유할지 묻는 과정이 무단 방치를 막고 냉장고 질서를 유지하는데 효과적이라고 느낀다.
- 질문 3) 버려질 뻔한 음식을 공유로 전환하거나 나눔을 받는 과정에서 자원 절약 및 공동체 유대감을 느낀다.
- 질문 4) 앱이 ‘빠르고 정확하다.’고 느껴지며, 중요한 알림을 놓치지 않을 거란 믿음이 있다.